

142. 环形链表 II

1. 问题

环形链表 II

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium (61.04%)	2792	0	-	-	0

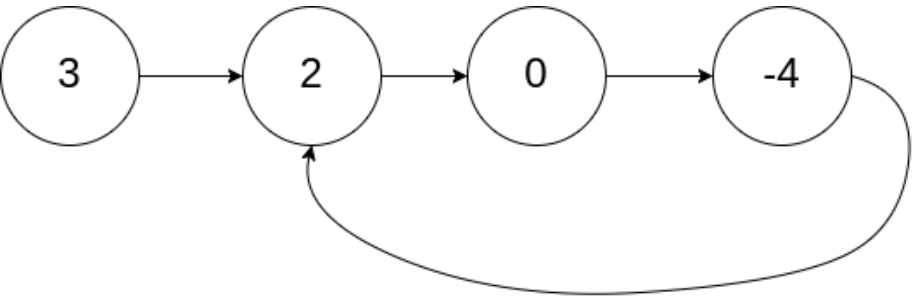
- Tags
- Companies

给定一个链表的头节点 `head`，返回链表开始入环的第一个节点。如果链表无环，则返回 `null`。

如果链表中有某个节点，可以通过连续跟踪 `next` 指针再次到达，则链表中存在环。为了表示给定链表中的环，评测系统内部使用整数 `pos` 来表示链表尾连接到链表中的位置（索引从 `0` 开始）。如果 `pos` 是 `-1`，则在该链表中没有环。注意：`pos` 不作为参数进行传递，仅仅是为了标识链表的实际情况。

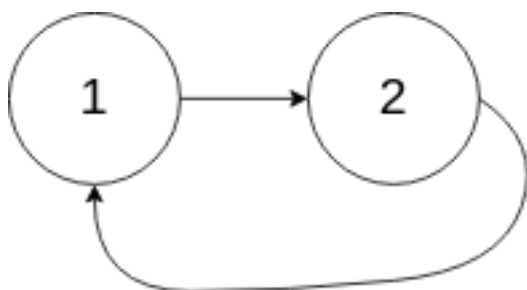
不允许修改 链表。

示例 1:



输入: `head = [3,2,0,-4]`, `pos = 1`
输出: 返回索引为 `1` 的链表节点
解释: 链表中有一个环，其尾部连接到第二个节点。

示例 2:

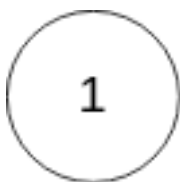


输入: `head = [1,2]`, `pos = 0`

输出: 返回索引为 0 的链表节点

解释: 链表有一个环, 其尾部连接到第一个节点。

示例 3:



输入: `head = [1]`, `pos = -1`

输出: 返回 `null`

解释: 链表中没有环。

提示:

- 链表中节点的数目范围在范围 `[0, 104]` 内
- `105 <= Node.val <= 105`
- `pos` 的值为 1 或者链表中的一个有效索引

进阶: 你是否可以使用 `O(1)` 空间解决此题?

[Discussion](#) | [Solution](#)

2. 方法

LeetCode 142: 环形链表 II (详细解析)

1. 问题理解

目标: 找到链表中环的入口节点, 如果链表中没有环, 则返回 `null`。

输入: 单链表的头节点 `head`

输出: 链表开始入环的第一个节点, 如果无环则返回 `null`

约束条件:

- 不允许修改链表结构
- 进阶要求：使用 $O(1)$ 空间复杂度解决

环的定义：

- 链表中某个节点的 **next** 指针指向链表中之前出现过的节点，形成一个环
- **pos** 表示环的入口节点在链表中的位置（索引从 0 开始），不作为输入参数

2. 普通人的思路（及遇到的困难）

思路一：使用哈希集合

想法：遍历链表，将访问过的节点存入哈希集合，第一个重复出现的节点就是环的入口。

步骤：

1. 创建一个哈希集合用于存储已访问的节点
2. 遍历链表，每遍历一个节点，先检查它是否已在集合中
3. 如果已存在，说明找到了环的入口，返回该节点
4. 如果不存在，将该节点加入集合，继续遍历
5. 如果遍历结束（遇到 **null**），说明没有环，返回 **null**

复杂度：

- 时间复杂度： $O(n)$
- 空间复杂度： $O(n)$ ，需要额外空间存储访问过的节点

缺点：不满足进阶要求的 $O(1)$ 空间复杂度。

3. 进阶解法（Floyd 判圈算法 - 快慢指针）

核心思想：使用快慢指针检测是否有环，然后利用数学关系找到环的入口。

两个关键步骤：

1. 检测是否有环：
 - 慢指针每次走一步，快指针每次走两步
 - 如果有环，两指针最终会在环内相遇
 - 如果无环，快指针会先到达链表末尾（值为 **null**）
2. 找到环的入口：
 - 当两指针在环中相遇后，将其中一个指针重置到链表头
 - 然后两个指针都改为每次走一步
 - 它们会在环的入口处再次相遇

为什么这样能找到环的入口？

这涉及一个数学证明：

假设：

- 从链表头到环入口的距离为 a
- 从环入口到相遇点的距离为 b
- 从相遇点顺时针到环入口的距离为 c
- 环的长度为 $b + c$

当快慢指针相遇时：

- 慢指针走了 $a + b$ 步
- 快指针走了 $a + b + n(b + c)$ 步（其中 n 是环的圈数）

由于快指针速度是慢指针的两倍，所以： $a + b + n(b + c) = 2(a + b)$

化简得： $a = c + (n-1)(b + c)$

特别地，当 $n=1$ 时， $a = c$ ，这意味着：从链表头到环入口的距离 = 从相遇点到环入口的距离

这就是为什么将一个指针重置到链表头后，两指针会在环入口相遇。

4. 示例演示

以示例 1 为例： $[3,2,0,-4]$ ， $pos = 1$ （尾部连接到索引为 1 的节点）

实际链表结构： $3 \rightarrow 2 \rightarrow 0 \rightarrow -4 \rightarrow 2 \rightarrow \dots$ （环从节点 2 开始）

1. 检测环：

初始：

$slow = 3, fast = 3$

第一次移动：

$slow = 2, fast = 0$

第二次移动：

$slow = 0, fast = 2$

第三次移动：

$slow = -4, fast = -4$ （相遇！）

2. 找环入口：

重置一个指针：

$ptr = 3, slow = -4$

同时移动：

$ptr = 2, slow = 2$ （再次相遇！）

找到环入口：节点值为 2 的节点（索引为 1）

5. 代码实现

```
/**
 * Definition for singly-linked list.
 * class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) {
 *         val = x;
 *         next = null;
 *     }
 * }
 */
public class Solution {
    /**
     * 找到链表中环的入口节点
     * 方法: Floyd 判圈算法 (快慢指针)
     * @param head 链表头节点
     * @return 环的入口节点, 无环则返回 null
     */
    public ListNode detectCycle(ListNode head) {
        // 处理边界情况
        if (head == null || head.next == null) {
            return null; // 空链表或只有一个节点的链表不可能有环
        }

        // 步骤一: 使用快慢指针检测是否有环
        ListNode slow = head;
        ListNode fast = head;
        boolean hasCycle = false;

        while (fast != null && fast.next != null) {
            slow = slow.next; // 慢指针每次走一步
            fast = fast.next.next; // 快指针每次走两步

            if (slow == fast) { // 如果相遇, 说明有环
                hasCycle = true;
                break;
            }
        }

        // 如果没有环, 直接返回 null
        if (!hasCycle) {
            return null;
        }
    }
}
```

```

// 步骤二：找到环的入口
// 将一个指针重置到链表头，两个指针同时以相同速度移动
ListNode ptr = head;

while (ptr != slow) {
    ptr = ptr.next;
    slow = slow.next;
}

// 当两个指针再次相遇时，相遇点就是环的入口
return ptr;
}
}

```

6. 代码解析

初始检查：

- 首先检查边界情况：空链表或只有一个节点的链表不可能有环

检测环：

- 使用快慢指针 `slow` 和 `fast` 检测链表是否有环
- 慢指针每次移动一步，快指针每次移动两步
- 如果有环，两个指针最终会相遇；如果无环，快指针会先到达链表末尾

找到环的入口：

- 在快慢指针相遇后，将一个指针 `ptr` 重置到链表头
- 然后 `ptr` 和 `slow` 指针同时以相同速度（每次一步）移动
- 根据数学证明，它们会在环的入口处相遇

7. 复杂度分析

时间复杂度： $O(n)$

- 最坏情况下，检测环的阶段需要遍历整个链表
- 寻找环入口的阶段最多再遍历整个链表一次
- 总时间复杂度为 $O(n)$ ，其中 n 是链表的长度

空间复杂度： $O(1)$

- 只使用了几个指针变量 (`slow`, `fast`, `ptr`)
- 不需要额外的数据结构来存储访问过的节点
- 满足进阶要求的 $O(1)$ 空间复杂度

8. 如何在面试中讲解

1. 确认问题理解：

- “这道题要求我找到链表中环的入口节点，如果链表没有环，则返回 null。进阶要求使用 $O(1)$ 空间复杂度。”
2. 提出思路：
 - “最直接的方法是使用哈希集合记录已访问的节点，但这需要 $O(n)$ 的额外空间。”
 - “为了满足 $O(1)$ 空间要求，我们可以使用 Floyd 判圈算法，也就是快慢指针技巧。”
 3. 讲解算法步骤：
 - “算法分两个步骤：首先检测链表是否有环，然后找到环的入口。”
 - “第一步：使用一个慢指针每次走一步，一个快指针每次走两步。如果有环，它们一定会在环内某点相遇。”
 - “第二步：相遇后，将一个指针重置到链表头，然后两个指针都改为每次走一步。它们会在环的入口处再次相遇。”
 4. 解释为什么有效：
 - “这个算法的有效性可以通过数学关系证明。简单来说，从链表头到环入口的距离等于从相遇点沿环方向到环入口的距离。”
 - “所以当两个指针同时从链表头和相遇点出发，以相同速度移动时，它们会在环入口处相遇。”
 5. 强调效率：
 - “这个算法的时间复杂度是 $O(n)$ ，空间复杂度是 $O(1)$ ，满足题目的进阶要求。”

通过使用 Floyd 判圈算法，我们优雅地解决了这个问题，既高效又符合空间限制。

找到具有 1 个许可证类型的类似代码